CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 28: Resolving truncation failures

Exam objective: Resolve structured output truncation failures by splitting large review tasks into smaller scoped API calls and merging resulting data structures, rather than increasing max_tokens beyond practical limits.

Q1 **A security review prompt asks the model to analyze an entire monorepo and return one giant JSON object of findings. Responses are frequently cut off mid-array. What is the best fix?**

A Keep increasing `max_tokens` until the full repository fits in one response

B Split the review into smaller scoped calls, such as per service or directory, then merge the results programmatically

C Ask for findings in natural language instead of JSON

D Lower temperature so the model writes faster

CORRECT ANSWER **B**

WHY Truncation is usually best solved by reducing response scope, not by chasing ever-larger token limits. Smaller calls produce complete structured outputs that can be merged reliably downstream.

Q2 A document extraction job returns a list of 2,000 clauses, but the model often stops halfway through the JSON. Why is simply raising `max_tokens` a poor long-term strategy?

A Higher `max_tokens` always reduces accuracy

B It does not address prompt-plus-output scaling limits, cost, latency, or reliability under variable input sizes

C The API forbids values above 100 tokens

D Larger outputs become invalid JSON by definition

CORRECT ANSWER B

WHY Increasing token budgets can be expensive, slow, and still unreliable for worst-case inputs. Breaking work into scoped units addresses the underlying scaling issue instead of treating symptoms.

Q3 A code-review application needs structured findings for 40 changed files. Which decomposition strategy is most appropriate?

A Send all 40 files in one request so the model sees complete context

B Review each file or logical file group in separate calls, then combine findings into a unified result set

C Ask the model to summarize only the first 10 files

D Return a plain-text essay instead of structured findings

CORRECT ANSWER B

WHY Per-file or logically grouped reviews bound output size while preserving enough local context. This yields complete, mergeable structured data without the truncation risk of a single oversized response.

Q4 After splitting a large review into smaller calls, what is the main responsibility of the client application?

A Nothing; the model will automatically merge prior outputs across requests

B Merge the resulting structured objects consistently, deduplicating or reconciling overlapping findings when needed

C Increase temperature to improve cross-call continuity

D Discard conflicting findings to save time

CORRECT ANSWER B

WHY Once work is decomposed, the application must aggregate results deterministically. Models do not automatically reconcile multiple independent API calls unless you explicitly orchestrate that logic.

Q5 A team reviews 500 support transcripts in a single batch request and asks for one JSON array of extracted issues. Truncation occurs unpredictably depending on transcript length. What is the best design change?

A Batch all transcripts together but ask for fewer fields

B Process transcripts in smaller batches with stable output size expectations, then concatenate validated results

C Compress the JSON by removing field names

D Ask the model to stop early when it feels the response is long

CORRECT ANSWER B

WHY Smaller, bounded batches make output size predictable and reduce truncation risk across variable inputs. Simply asking for fewer fields or early stopping does not guarantee completeness.

Q6 **A developer argues that one large call is easier because it preserves global context. What is the best response when truncation is already causing incomplete structured output?**

A Global context is more important than completeness, so keep the large call

B Completeness and parseability are prerequisites; preserve necessary context by scoping intelligently, not by overloading one response

C Switch to unstructured output so truncation is less obvious

D Use a random sample of the input instead

CORRECT ANSWER **B**

WHY A complete structured result is more valuable than an idealized but truncated global analysis. You can preserve important context through thoughtful partitioning while still keeping each response within practical bounds.

Q7 **A review job returns partial JSON because the model runs out of output tokens. Which metric should most directly guide your redesign?**

A CPU utilization on the client machine

B The size and variability of the expected output per call

C The number of developers on the team

D The alphabetical order of input files

CORRECT ANSWER **B**

WHY Truncation is driven by output volume relative to token limits, so the expected result size per call is the key design variable. Understanding that distribution helps you choose stable, bounded scopes.

Q8 You split a giant extraction task into smaller calls but still see occasional truncation on unusually dense inputs. What is the best next step?

A **Further reduce scope size or cap items per call, then continue merging results**

B Abandon structured output entirely

C Double temperature and hope shorter answers emerge

D Retrain the downstream parser to accept incomplete JSON

CORRECT ANSWER A

WHY If some partitions remain too large, the correct response is finer-grained decomposition. Accepting incomplete JSON or abandoning structure treats symptoms and weakens reliability.

Q9 A client merges findings from per-file reviews and notices duplicate issues where the same defect spans two files. What is the best merge strategy?

A Keep every duplicate to avoid missing anything

B **Implement deterministic deduplication rules using file paths, line ranges, issue types, or normalized signatures**

C Randomly keep one of the duplicates

D Ask the model to remember all prior calls without sending them

CORRECT ANSWER B

WHY Once you decompose work, merging must include reconciliation logic for overlapping results. Deterministic deduplication preserves completeness without inflating counts.

Q10 A legal-tech pipeline extracts sections from a 300-page filing. The model truncates when asked for all extracted sections at once. Which redesign best fits the objective?

A **Process page ranges or section ranges separately and assemble the final structured object afterward**

B Increase `max_tokens` and set a stronger warning in the prompt

C Ask for plain text first and convert it to JSON with regex

D Only extract headings, not bodies

CORRECT ANSWER **A**

WHY Partitioning large documents into logical or page-based segments addresses output-size constraints directly. It also makes failure recovery easier because you can retry only the affected segment.

Q11 Why is splitting large structured reviews often better operationally even when larger token limits are technically available?

A **Smaller calls are easier to retry, monitor, validate, and parallelize**

B Large-token calls are forbidden by JSON standards

C Models always reason worse on short inputs

D Parallelization is impossible with smaller tasks

CORRECT ANSWER **A**

WHY Decomposition improves resilience and system design, not just truncation avoidance. Smaller units support selective retries, easier validation, and more predictable latency profiles.

Q12 A product team wants a single API call because "merging JSON is annoying." What is the best architectural counterargument?

A Merging is unnecessary if you ignore truncation

B Client-side merging is simpler and safer than handling incomplete, invalid, or silently truncated results from oversized responses

C JSON cannot be merged programmatically

D Structured output should only be used for toy problems

CORRECT ANSWER B

WHY A reliable multi-call design with deterministic merging is preferable to a fragile single-call design that intermittently fails. Merge logic is usually straightforward compared with diagnosing partial outputs in production.

Q13 A quality-evaluation task asks the model to score 1,000 summaries and explain each score. Which change most directly reduces truncation risk?

A Evaluate the summaries in smaller batches and store scores incrementally

B Ask for shorter explanations but still process all 1,000 at once

C Replace the explanations with emojis

D Use a higher temperature to encourage concise responses

CORRECT ANSWER A

WHY Batching reduces both output size and variance per request. Smaller explanations may help somewhat, but keeping all 1,000 items in one response still leaves substantial truncation risk.

Q14 A developer says, "If a response truncates, we'll just parse whatever JSON fragment we got." Why is this poor practice?

A **Partial structured output can silently drop findings, corrupt arrays, and produce misleading downstream state**

B JSON fragments are always larger than full JSON

C Fragment parsing is slower than SQL

D Truncation only affects human readability, not machine correctness

CORRECT ANSWER A

WHY Truncation is not merely cosmetic; it can produce incomplete or malformed data that distorts downstream behavior. Reliable systems should prevent or recover from truncation, not normalize it.

Q15 You need to preserve some cross-document context while avoiding truncation. Which approach is best?

A Keep one giant call because any split loses all context

B **Use hierarchical processing: smaller scoped extraction calls followed by a separate aggregation or synthesis step over the compact results**

C Turn off structured output

D Ask the model to compress the original inputs internally

CORRECT ANSWER B

WHY Hierarchical workflows retain high-level context without demanding one enormous structured response. First extract bounded local results, then synthesize or reconcile them in a later step with a much smaller intermediate payload. --